

Received 5 February 2025, accepted 6 March 2025, date of publication 10 March 2025, date of current version 20 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3550009

RESEARCH ARTICLE

Measurement of Anti-Debugging Techniques on the Windows and Linux Operating Systems for the Intel x86_64 Architecture

AUSTIN NORBY¹, BHASKAR P. RIMAL², (Senior Member, IEEE), AND BRAMWELL BRIZENDINE³

¹Bogart Associates of Northern Virginia, Reston, VA 20190, USA

²Department of Computer Science, University of Idaho, Moscow, ID 83844, USA

³Department of Computer Science, The University of Alabama in Huntsville, Huntsville, AL 35899, USA

Corresponding author: Bhaskar P. Rimal (bhaskar.rimal@ieee.org)

Publication of this article was funded by the University of Idaho, Moscow, ID, USA, through Open Access Publishing Fund.

ABSTRACT Malware authors and software protection frameworks often use anti-debugging techniques to hinder understanding of the underlying code. Companies use anti-debugging techniques to prevent intellectual property, including music, movies, and games, from being stolen or abused by malicious actors. In addition, the encryption of program data or private data can also be classified as an anti-debugging technique because it requires additional effort to reverse engineer or debug it. Malware authors also use these techniques to prevent their software from being analyzed, detected, or blocked. While anti-debugging techniques are not novel, there is minimal research on their performance. This paper conducted novel research on commonly used anti-debugging techniques by measuring their performance on both the Windows and Linux operating system while also creating performance data about the most common anti-debugging techniques, including implementations and statistical measures for six categories of anti-debugging techniques. The statistical results describe a low overhead for the API tests, timing tests, and debug register tests. As expected, the memory encryption test had the highest overhead. Lastly, the paper discusses these results and their statistics per technique and includes code samples for future research.

INDEX TERMS Anti-debugging, code obfuscation, debugger, malware analysis, memory encryption, OpenSSL.

I. INTRODUCTION

There are many reasons for wanting to prevent third parties from debugging or reverse engineering an application. Some of the reasons could be to protect proprietary information, hide secure credentials, and protect intellectual property. Malware development also uses anti-debugging techniques to evade detection and hinder analysis.

Anti-debugging checks, implementations, and research have existed since at least the late 1990s [1]. The concept is not operating system or architecture-dependent. However, certain techniques may be. For example, checking

IsDebuggerPresent() in the Windows operating system is different than checking /proc/\$pid/status, but both return information related to the debugger (if it exists) for the current process [2], [3], [4]. In addition, calling a function that executes a single instruction, IsDebuggerPresent(), will likely perform faster than reading and parsing the file, /proc/\$pid/status, even though they return the same information.

Anti-debugging techniques are often used in copy-protection schemes associated with proprietary information and intellectual property. Software protection has a few different levels of sophistication. The techniques range from software registration using identification and serial numbers to complex, sophisticated binary packing of the executable, including network authentication of the

The associate editor coordinating the review of this manuscript and approving it for publication was Ali Kashif Bashir¹.

software [5], [6], [7], [8]. This protection can be seen in many industries, including video gaming [9].

Another common use case for anti-debugging is within the malware. Malware authors will use these techniques to hinder malware analysis so it stays viable longer and can affect more victims [10], [11]. According to [10], [11], and [12], approximately 40% of malware uses some anti-debugging technique. Malware development is the malicious side of the problem, but it also applies to the blue team and malware analysis. Malware analysts must understand and overcome any anti-debugging or anti-reverse engineering techniques used in malware. The same research, [10], created a simple heuristic for detecting malware as well based on a risk score, including the technique, prevalence, and impact. The offensive and defensive sides of anti-debugging techniques build on each other over time.

Most research focuses on the hindrance, implementation, and defeat of anti-debugging techniques, but there is limited research on the performance aspect of anti-debugging techniques. This paper does not focus on identifying new techniques but, on analyzing current techniques from a performance perspective that has not been pursued before. Not all techniques are the same; some require significantly more resources to implement and defeat than others. Measuring the execution overhead of these techniques is a step towards better understanding the impacts of anti-debugging techniques. When studied together, anti-debugging, anti-virtualization, anti-dumping, anti-tracing, copy-protection, tamper-resistant, etc., lead to a better understanding of the field of anti-debugging writ large [13]. This paper starts by measuring the execution times of common techniques on both the Windows and Linux operating systems for comparison between techniques and operating systems to understand anti-debugging performance better. This paper does not implement any kernel-mode drivers to force execution into a single-threaded execution of the technique. These techniques are executed in user mode, which is more representative of the timings of execution in reality, and are impacted by the operating system scheduler and context switching [14].

This paper contributes novel research to the anti-debugging field in the form of code, results, and data visualization by measuring the execution overhead of common anti-debugging techniques and publishing the research code on an open-source platform to help others within the academic and professional industry. Within this paper, we tackled implementation challenges as not all anti-debugging techniques are published with working source code nor freely released into the open-source ecosystem. Another issue we tackled in this paper is producing data about anti-debugging performance. This data did not exist for either operating system. They required us to generate the data to study the performance impact of these techniques individually and across operating systems. We proposed these algorithms to measure the most common techniques across the most common operating systems to generate the most meaningful data for analysis. The novelty of this research comes from the

combination of performance measurement, anti-debugging techniques, and measuring and comparing the data across operating systems. This paper aims to analyze novel data that informs users of anti-debugging techniques and the impact of performance on their code.

Towards this end, the key contributions of this paper are summarized as follows:

- Novel anti-debugging measurement and logging source code for measuring and executing six categories of anti-debugging techniques across Windows and Linux operating systems including `IsDebuggerPresent()` (Windows), Checking `IsTracerPid` value in `/proc/self/status` (Linux), Checking for known parent process names in a deny list, Memory Encryption, Timing Analysis, Self-debugging using `ptrace()` (Linux), Self-debugging using a child process (Windows), and Code obfuscation.
- Performance data from twelve different anti-debugging measurement and logging algorithms across Windows and Linux operating systems on the Intel x86_64 Architecture.
- This paper generated data through the novel anti-debugging measurement and logging algorithms that previously did not exist. Each anti-debugging technique will have different amounts of performance overhead. The raw data is necessary for statistical analysis and informs users of anti-debugging techniques and the impact of these techniques on their code.
- We conducted statistical analysis and visualization of the performance overhead for each anti-debugging algorithm. These results inform the users of anti-debugging techniques of the performance impact of each technique both individually and across operating systems.
- For reproducibility and validity of our concept, we published source code that generates data visualization for measuring anti-debugging technique duration.

The remainder of this paper is structured as follows. Section II discusses the related work for anti-debugging techniques on Linux and Windows operating systems and briefly addresses the connection with anti-virtualization techniques. Section III focuses on the methodology for this paper, including the algorithms that measure the anti-debugging technique and log the data. Section IV discusses the results of the measurements, including data tables, plotted distributions on graphs, and comparisons of techniques between operating systems and the baseline algorithm. Finally, Section V concludes the paper and discusses future work aligned with this research.

II. RELATED WORK

Anti-debugging research focuses (e.g., [15]) mostly on malware analysis and copy-protection schemes. Some foundational techniques focus on timing analysis, debug register checking, and debugger identification. Once the debugger has been identified then a penalty can be enacted, such as process or debugger termination, deception, or execution of alternate code paths as a result.

In [16], authors provided an efficient description of many techniques across six categories of anti-debugging techniques: API-based anti-debugging techniques, exception-based anti-debugging techniques, direct process, and thread detections, modified code detection, hardware and register detection, timing-based anti-debugging. Each category has one or more techniques to explain the anti-debugging technique, sometimes with proof-of-concept code. The API-based anti-debugging techniques focus on Windows API calls such as `IsDebuggerPresent()`, `CheckRemoteDebuggerPresent()`, and `NtQueryInformationProcess()`. The exception-based anti-debugging techniques focus on manipulating the debugger and operating system such that when an exception is handled before the exception handler (Windows) or signal handler (Linux) handles the exception, it is implied a debugger is present. This implication is valid because, in these operating systems, the exception/signal handlers would normally handle the exception when a debugger is not present. Therefore, when they do not handle the expected exception, something else must have handled it, and therefore the presence of a debugger is implied.

In [17], the authors discussed 79 different techniques as part of six larger categories and 17 sub-categories similar to the technique mentioned above categories. The authors developed a programmatic solution to invasively defeat anti-debugging techniques using modern techniques and defeats, including their anti-debugging techniques and defeats. Their solution outperforms other debuggers, including Windbg, Ollydbg, and IDA Pro. However, it does not look into the performance of these techniques at all.

In [18], authors implemented an encrypter packer that uses anti-debugging, anti-memory dumping, and general anti-analysis techniques to prevent reverse engineering. We have created an artifact that measures the startup execution time to display the differences between a regular binary and an encrypter-packed variant. Adding additional execution instructions, including encryption, will significantly increase the program's startup time, as demonstrated in the research. Still, the researchers also confirm the anti-debugging and anti-dumping features are functional. Other research related to packing executables is software protection (e.g., [6]) which develops a framework for protecting the code and minimizing the amount of time it is exposed. This can be done by intercepting function calls to pass through execution to an injected DLL that protects the code. This process has a wide range of performance impact from 4% to 177% depending on the techniques as seen in [6].

Other research focused on applications for Android-based systems. Mobile applications and devices are a desirable target for legitimate and malicious software due to the constant user interaction and storage of important information. Some techniques include code obfuscation, emulator detection, self-debugging, time difference, and analyzing parent processes. Code obfuscation uses function name mangling and unnecessary instructions to obfuscate the

code's intention. The emulator detection looks for known differences between a real, hardware-based system and known values that only exist in an emulator. The emulator detection is not designed to be perfect, but its existence is enough to support general anti-debugging implementations.

Closely related to anti-debugging techniques are anti-virtualization techniques [11], [12]. Malware analysis can be done in a virtualized environment to contain the malware and prevent accidental damage to physical systems or unintended propagation on a personal, research, or corporate network. The anti-virtualization techniques can hinder debugging in a similar way to anti-debugging checks specifically. In [12], the authors constructed a sophisticated and exhaustive suite of tests to identify possible anti-virtualization techniques using instructions with particular inputs. These groups of instructions and inputs are called "pills" and can be used to identify the presence of execution within a virtualized environment. The authors completed an exhaustive analysis of the Intel x86 instruction set, looking for possible "pills," and then executed the code on both a physical and virtual system to compare the differences between the execution contexts.

In [19], authors developed a kernel driver that acted as the Virtual Machine Monitor (VMM). Normally, the VMM is located outside the operating system in the hypervisor for hardware virtualization. In some literature, the VMM is located at "ring -1". This expands upon the idea that the Windows operating system kernel exists in ring 0 and user mode exists in ring 3. The use of only these two rings is a commonly cited example of a problem with the security architecture of the Windows operating system. Using this idea, the authors developed a kernel module to prevent access to the Process Control Block (PCB) and the CR3 register of other processes. These pieces of information are necessary for debugging, and thus, the target software cannot be debugged [19]. [19] differs from this proposed research using a kernel driver; all techniques studied and measured here exist solely in Ring 3 (User-mode).

Self-debugging lets a process attach to itself as a debugger, which prevents any other process from attaching as a debugger. This can even be improved upon by having the protected application debug the child process or self-debugger, and the self-debuggee then debug the protected process as expected. This circular dependency significantly hinders breaking into or attaching to either process to gain control of either [2], [16], [20].

In [21], authors looked into kernel-based solutions. This paper looked at improving the debugging capabilities for a Linux-based system by using "uprobes" and "kprobes" libraries and the commonly used `ptrace` syscall to move the debugging engine into the kernel. Moving the heavy lifting of debugging a process to the kernel provided benefits such as performance increases and direct access to process, thread memory, and registers, and it was more difficult for malware to locate based on current techniques.

TABLE 1. Summary of related work.

Category/Technique	Summary
Anti-Debugging Techniques	There are dozens of anti-debugging techniques that have been discovered and each one requires a specific implementation within a specific operating system to detect the presence of a debugger. The authors of apate [17] identified 79 techniques. Other papers gave implementation details such as [1] for Linux anti-debugging techniques specifically and [5]–[8] for Windows techniques. Anti-debugging is mostly used in malware to avoid detection and the research in detection and defeating anti-debugging techniques is strongly correlated to analyzing malware.
Performance	Performance related to anti-debugging techniques is often overlooked or added as an afterthought. [14] describes in detail how to measure the performance of any program but it is not related to anti-debugging techniques in general. The closest research to performance related to anti-debugging techniques is the software protection paper, [6], that includes performance overhead for their software protection framework.
Software Protection	Software protection also makes use of anti-debugging techniques for a less malicious purpose and focuses on protecting intellectual property or copyright information. In much the same way as malware does not want to be analyzed, protected software also implements these checks to avoid being reverse-engineered and the sensitive information disclosed.

The artifact, plutonium-dbg, used ptrace() when necessary for some of its functionality. It also used the hooking capabilities of uprobes and kprobes to handle and modify states that ptrace() would normally not be able to handle. In addition, using plutonium-dbg as a Loadable Kernel Module (LKM) removed the ability to check if the process was being debugged using ptrace() and potentially allowed for multiple debuggers, which defeats the self-debugging anti-debugging technique. This paper does not focus on any kernel driver implementations for anti-debugging techniques or performance measurement.

The next technique is measuring the time differences during execution. If the time difference is too large, it will likely be debugged. Lastly, if the parent process is a known debugger, then the anti-debugger can terminate the process using just that information. More research on the Android operating system has focused on cryptography and how to prevent an attacker that has attached to a process or is debugging the system from retrieving secure keys. In [22], the authors developed an intermediary virtual machine that interfaced between the mobile application and the native run-time. The intermediary focused on anti-debugging techniques, like timing analysis, and used device fingerprinting and memory encryption to hinder reverse engineering efforts. In [23] the authors used hooking and self-debugging to validate the code integrity and protect the code from dynamic and static analysis in the Android operating system.

Another technique (e.g., [16], [24]) uses Structured Exception Handling (SEH), or Vectored Exception Handling, to check for the presence of a debugger. This technique works by only executing code if the debugger is not present. If the debugger is present, the execution will fail, but if the debugger is not, the code will clean up and continue execution as normal.

Not all work is focused on improving anti-debugging efforts, however. In [25], authors have published an artifact that can bypass some anti-debugging checks and evaluate the original program despite anti-debugging efforts. The artifact is called x64Unpack, and it uses a combination of CPU emulation, function hooking, memory tracking

(including a custom stack), custom exception handling, and more to evaluate a binary. The research focused on Windows PE files and evaluated other packers, such as VMProtect, which is considered highly sophisticated. The x64Unpack tool could emulate the unpacking of a binary that was packed using VMProtect and run the original code despite the built-in anti-debugging checks used by VMProtect, including API calls, timing analysis, integrity checks, memory permissions, and thread management. This work is equally as interesting as developing anti-debugging techniques because it is effectively an anti-anti-debugging technique. Similarly, in [11], techniques were developed to modify the execution environment to fool malware into thinking it's on a virtualized or debugging-capable system. Further, in [26], authors created a rule-based system to neutralize the anti-debugging techniques in a file. Much like a static signature search, the bytes could be identified, added to the rule-based system, and then checked statically against the sample. This has the benefit of not being dynamically determined by the malware during runtime. Table 1. summarizes the related work.

III. METHODOLOGY

For this work, we have implemented and measured the overhead CPU cost of the following popular anti-debugging techniques [2], [3], [4], [22], [27], including a) API Debugger Check (IsDebuggerPresent() or /proc/self/status), b) Validate Parent Process, c) Memory Encryption, d) Timing analysis, e) Self-debugging, and f) Code obfuscation. Some checks are not expected to contribute much overhead, such as calling OS APIs, like IsDebuggerPresent(), and others will contribute significantly more overhead, like memory encryption. The classification and algorithm list is detailed in 2

We implemented techniques and added them to a base program for accurate measurements of the execution overhead for each technique. The base execution will perform millions of iterations of thousands of operations to provide enough work and data to measure the overhead effectively. In addition, these programs will be adapted to the Windows and Linux operating systems. Each operating system is a different target for malware and copy-protection

TABLE 2. Anti-debugging classifications and algorithms.

Classification	Algorithm (Windows)	Algorithm (Linux)
OS API	IsDebuggerPresent()	/proc/\$pid/status
Validate Parent Process	CreateToolhelp32Snapshot and GetCurrentProcessId	getppid() and /proc/\$ppid/cmdline
Memory Encryption	BCrypt	OpenSSL
Timing Analysis	timeGetTime()	time()
Self-Debugging	Child Debugging	ptrace()
Code Obfuscation	Loop Unrolling, Function Pointer Arithmetic, Bogus Functions	Loop Unrolling, Function Pointer Arithmetic, Bogus Functions

code. Understanding the computational overhead for each operating system and technique is beneficial. In addition, some techniques are not identical across operating systems: `IsDebuggerPresent()` is different than reading the `/proc/self/status` virtual file.

The measurement will focus on the length of time of execution. The measured values from the anti-debugging version of the program will be compared to the measurements of the unaltered version and visually represented for the readers. Before calculating statistics, data more than 3 standard deviations from the mean is removed to avoid undue bias from outliers. For this paper, standard statistical measurements for the data include mean, median, and standard deviation to quantify and compare results across techniques. In addition, while the Linux results were measured and graphed in nanoseconds, the tables have been converted to Ticks to make comparison between the results per operating system easier to understand. This is also why the Linux timing values include decimal places as the conversion between ticks and nanoseconds is 1 Tick per 100 nanoseconds.

Each operating system was run within the VMWare Workstation given the same amount of resources: 4GB of memory and 2 vCPUs. These machines were running at the same time as virtualization. These virtual machines were not competing for resources on the host OS with a 12th Gen Intel i7-12700k processor, which includes 12 CPU Cores, 20 Logical Processors, and 128GB of memory. In addition, the VMs were run separately to eliminate any chance of competing with each other for resources in addition to competing with the host for resources. The Windows guest operating system is Windows 10 1703 $\times$ 86_64, and the Linux guest operating system is Ubuntu 22.04.1 LTS running the x86_64 5.15.0-56 kernel.

A. BASELINE CODE

For the Windows and Linux code, we executed one million iterations of vector data manipulation, which will require some time and allow for enough overhead such that each loop can be measured, and each type of test will have approximately one million data points for analysis. The baseline code is shown in Algorithm 1.

This structure is apparent in all Windows and Linux research codes in the same way, with possible differences as needed for each anti-debugging technique. The `AntiDebugging_Time` functions are used to measure the number of ticks of time that occurred between the beginning

Algorithm 1 Baseline Algorithm

```

StartFullTime = 0           ▷ Full test start time
StartSingleTime = 0         ▷ Single test start time
EndFullTime = 0             ▷ Full test end time
EndSingleTime = 0           ▷ Single test end time
StartFullTime = StartTimer() ▷ Read current time and
                             store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
computational cost
    StartSingleTime = StartTimer() ▷ Read current time
    and store in StartSingleTime
    TimeIntensiveOperations(byte) ▷ Perform math
    operations that take time on each byte
    EndSingleTime = StopTimer() ▷ Read current time
    and write times to log file (Single)
EndFullTime = StopTimer() ▷ Read current time and write
times to log file (Full)

```

and end of the individual execution loop. The final result is the total amount of time that the entire one million loops took. Thus, each set of data will have one million and one record, with the final record being the total amount of time for every loop. The naming conventions changed within the Linux sample, but the same test was conducted within the bounds of operating system differences. In addition, for the Linux tests, the timer was renamed to `ResearchTimer`.

The timing functions use Microsoft Windows' `QueryPerformanceCounter` API. The start function captures the initial value, and the stop function captures the final value and returns the difference as the number of ticks that have occurred while the code being measured has been executed. On the Linux system, `clock_gettime()`. This timer is used for high-precision timing measurements and measures the CPU time of execution.

B. ANTI-DEBUGGING TECHNIQUE CODE OVERVIEW

1) OS API ANTI-DEBUGGING CHECK

The first anti-debugging technique will be checking if the process is being debugged using APIs/system information. For the Windows operating system, we will be using the function `IsDebuggerPresent()`. For the Linux operating system, we will be reading the `/proc/self/status` file and checking if a debugger is present. These functions will be called before each loop of work to ensure no debugger has been attached later in the execution.

The code for Windows will use the API call and check the boolean result. The implementation is illustrated in Algorithm 2, and for Linux, see Algorithm 3.

Algorithm 2 Windows API IsDebuggerPresent()

```

StartFullTime=0           ▷ Full test start time
StartSingleTime=0         ▷ Single test start time
EndFullTime=0             ▷ Full test end time
EndSingleTime=0           ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
    store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
    computational cost
    StartSingleTime=StartTimer() ▷ Read current time
    and store in StartSingleTime
    IsDebuggerPresent()      ▷ Windows API Debugger
    Check
    TimeIntensiveOperations(byte) ▷ Perform math
    operations that take time on each byte
    EndSingleTime=StopTimer() ▷ Read current time
    and write times to log file (Single)
EndFullTime=StopTimer() ▷ Read current time and write
    times to log file (Full)
  
```

Algorithm 3 Linux IsTracerPidZero

```

StartFullTime=0           ▷ Full test start time
StartSingleTime=0         ▷ Single test start time
EndFullTime=0             ▷ Full test end time
EndSingleTime=0           ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
    store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
    computational cost
    StartSingleTime=StartTimer() ▷ Read current time
    and store in StartSingleTime
    OpenFile('/proc/self/status') ▷ Open status file for
    process
    While readline      ▷ While reading lines from the file
        Tag, Value = line.split() ▷ Split line into tag and
        value
        if tag == "TracerPid" and value != 0 ▷ Nonzero
        value of the TracerPid implies debugger
            NOP          ▷ Executing a
            penalty would skew results, however the check is part
            of the technique for measurement
            TimeIntensiveOperations(byte) ▷ Perform math
            operations that take time on each byte
            EndSingleTime=StopTimer() ▷ Read current time
            and write times to log file (Single)
    EndFullTime=StopTimer() ▷ Read current time and write
    times to log file (Full)
  
```

As previously stated, we can see the similarity between the baseline code and the testing code. The only difference in

this technique is the usage of the IsDebuggerPresent API to check if a debugger is present. Linux does not have an equivalent API call for IsDebuggerPresent(), so the code opens the `/proc/self/status` file and reads the value of "TracerPid:". If the TracerPid value is non-zero, then it knows it is being debugged.

2) PARENT PROCESS ANTI-DEBUGGING CHECK

The second technique is validating the parent process. The code will find its Parent Process ID and lookup, based on the ID, the image name of the parent process if the parent process matches a known debugger or other simple naming checks (e.g., contains the string: "dbg" or "debug"). This code will be executed before every loop to prevent a parent process from renaming itself to bypass a single check.

For the Windows implementation, the code gets a list of all of the processes and finds the parent process for the currently running program. This code uses the `CreateToolhelp32Snapshot()` and `GetProcessId()` Windows API functions to resolve the parent process of the current process. Once the algorithm finds the parent process, the algorithm compares the parent process name to a list of known debuggers, and if the parent process matches, then a debugger is detected. The denied list of process names for the Windows test includes dbg, windbg, ollydbg, debug, monitor, watch, x96dbg, x64dbg, and enghost. As for Linux, the denied list of parent processes includes gdb, lldb, kdbg, valgrind, strace, and ltrace.

Algorithm 4 GetParentProcessName

```

StartFullTime=0           ▷ Full test start time
StartSingleTime=0         ▷ Single test start time
EndFullTime=0             ▷ Full test end time
EndSingleTime=0           ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
    store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
    computational cost
    StartSingleTime=StartTimer() ▷ Read current time
    and store in StartSingleTime
    Pid = GetPID()          ▷ Get PID of current process
    Ppid = GetPPID()        ▷ Get Parent PID of current process
    ParentName = GetProcessName(Ppid) ▷ Get the
    process name from the parent PID
    If ParentName in DenyList ▷ Check if parent process
    name is in deny list
        NOP                ▷ Executing a
        penalty would skew results, however the check is part
        of the technique for measurement
        TimeIntensiveOperations(byte) ▷ Perform math
        operations that take time on each byte
        EndSingleTime=StopTimer() ▷ Read current time
        and write times to log file (Single)
    EndFullTime=StopTimer() ▷ Read current time and write
    times to log file (Full)
  
```

The parent process check was implemented using Algorithm 4. Each technique required a list of possible image names for debuggers or programs that could attach to the running program. For example, the `strace()` command is not a debugger but attaches itself as a debugger that can be detected. The Linux code uses the `getppid()` function to get the Parent PID of the process and open up the `/proc/$PPID/cmdline` file for reading. If any deny-list value is found contained within that file, we can assume a debugger exists and it's been detected.

3) MEMORY ENCRYPTION

The third technique is memory encryption. For this technique, we will encrypt the function that does work and dynamically decrypt it before every loop of execution. This type of anti-debugging technique is used to protect important code. Still, it requires a high overhead due to the amount of additional code needed to execute the work and memory storage for the encrypted and unencrypted data.

The Windows implementation uses the BCrypt Encryption algorithms and interfaces. This requires building up the required data structures before any encryption is performed. These settings and structures are built outside of the timing functionality as they only need to be performed once for an entire program. The timing begins with decrypting the encrypted values in memory, performing the standard mathematical operations on the data, and re-encrypting the data. This would follow the process of other anti-debugging techniques that encrypt and decrypt just in time for use and only for the use of the data. The implementation for the memory encryption test is shown in Algorithm 5.

Algorithm 5 Memory Encryption

```

StartFullTime =0           ▷ Full test start time
StartSingleTime=0         ▷ Single test start time
EndFullTime=0             ▷ Full test end time
EndSingleTime=0           ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
                           store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
computational cost
    StartSingleTime=StartTimer() ▷ Read current time
and store in StartSingleTime
    DecryptData(byte)          ▷ Decrypt data for use
    TimeIntensiveOperations(byte) ▷ Perform math
operations that take time on each byte
    EncryptData(byte)          ▷ Encrypt data for storage (when
not in use)
    EndSingleTime=StopTimer()  ▷ Read current time
and write times to log file (Single)
EndFullTime=StopTimer() ▷ Read current time and write
times to log file (Full)

```

Unlike the Windows code, we used an OpenSSL library that encrypted the data using AES-256-CBC for the Linux Memory Encryption algorithm. This information was

decrypted before manipulation and re-encrypted afterward, just like the Windows version.

4) TIMING ANALYSIS

The fourth technique will be timing analysis. Using the base program's timing, we will create a threshold of acceptable run-time range and check that this range is not violated at each iteration of the loop for work. If the timing threshold is violated, it is likely the program was being debugged, and therefore, other actions should be taken. In this research, additional action will not be taken, and we will measure the overhead this timing check adds to the code.

The Windows implementation of this code uses a different timing mechanism than the anti-debugging measurement code. This timing technique uses built-in Windows timers that can be set to a timing precision and then measured pre- and post-execution. For this implementation, the resolution was set to 100ms, and a debugger was detected if the code execution took longer than 500ms (Threshold). This technique is designed to catch breakpoints, sleep, or human analysis, and the threshold is set high enough that even on the slowest runs, it would be exceptionally hard for the code to take that long and accidentally pass the threshold, thereby generating a false positive, see Algorithm 6.

Algorithm 6 Timing Analysis

```

StartFullTime =0           ▷ Full test start time
StartSingleTime=0         ▷ Single test start time
EndFullTime=0             ▷ Full test end time
EndSingleTime=0           ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
                           store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
computational cost
    StartSingleTime=StartTimer() ▷ Read current time
and store in StartSingleTime
    TimingAnalysisStartTime = StartTimer() ▷ Get time
for anti-debugging check
    TimeIntensiveOperations(byte) ▷ Perform math
operations that take time on each byte
    TimingAnalysisStartTime = StopTimer() ▷ Stop
anti-debugging timer
    //If the computation takes too long, it is likely being
debugged (greater than the threshold)
    If TimingAnalysisStopTime - TimingAnalysisStart-
Time > Threshold
        NOP
    EndSingleTime=StopTimer() ▷ Read current time
and write times to log file (Single)
EndFullTime=StopTimer() ▷ Read current time and write
times to log file (Full)

```

We use different objects and a similar but different conditional for measurements for the Linux version. For this specific timing test, we used the `clock_t` objects that measure seconds but not a fraction of a second. If the sample took

Algorithm 7 Windows Self-Debugging Parent

```

StartFullTime=0           ▷ Full test start time
StartSingleTime=0         ▷ Single test start time
EndFullTime=0             ▷ Full test end time
EndSingleTime=0           ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
computational cost
    StartSingleTime=StartTimer() ▷ Read current time
and store in StartSingleTime
    Pid = GetPID()
    Child = CreateProcess("Selfdebuggingchild.exe",
Pid)           ▷ Create special child to attach to parent
    TimeIntensiveOperations(byte) ▷ Perform math
operations that take time on each byte
    KillProcess(Child)      ▷ Kill child process debugging
parent
    EndSingleTime=StopTimer() ▷ Read current time
and write times to log file (Single)
EndFullTime=StopTimer() ▷ Read current time and write
times to log file (Full)

```

longer than 1 second (Threshold) to execute for this test, it was classified as debugged.

5) SELF-DEBUGGING

The sixth technique is self-debugging. We will take the simple approach and only spawn a child process that acts as the self-debugger rather than creating a circular dependency between the self-debugger and the protected program. The self-debugger will implement the necessary debugging interfaces based on the operating system and will attach to the program but in no way functionally interfere with execution.

For the Windows implementation, this required using the Windows debugging interface as part of the child process and using the `CreateProcess` Windows API in the parent to spawn the child process. Spawning the child process passes in the necessary information for the child to be able to debug the parent process, namely, the parent process ID (PPID). Algorithm 7 shows the parent process or test code.

Once the child process is spawned, it uses the debugging interface to attach to the parent via Process ID (PID) and continue execution until the parent forcibly terminates the child process. The child process is shown in Algorithm 8.

This code requires more use of the Windows API due to interfacing with the debugger interface. The child process attaches to the parent process by providing the PID to the `DebugActiveProcess()` function. Once the child is actively debugging the parent, it sets a configuration value via `DebugSetProcessKillOnExit` to false so that when the child

Algorithm 8 Windows Self-Debugging Child

```

Ppid = argv[1]           ▷ Get Parent PID from args
DebugActiveProcess(Ppid) ▷ Attach as debugger to
parent process
While True               ▷ Infinite Loop
    If WaitForDebugEvent() ▷ Let the process run until
there is a debug event
        ContinueDebugEvent() ▷ When there is a debug
event, continue execution

```

stops debugging the parent process, it is NOT killed. Lastly, the child waits for a debug event from the parent process, and whenever an event is captured, it continues execution so that the parent will execute like normal.

Linux offers an interesting twist to self-debugging that Windows does not. Within Linux, the `ptrace()` capability can truly debug itself. Using the `PTRACE_TRACEME` value when calling `ptrace()` within a process, the process becomes its debugger and debugged, and no other processes can attach to it. This technique is much smaller and quicker than the Windows technique. However, the catch is that once a process is debugging itself, it cannot stop. The only way to stop is to terminate the process [28], [29]. Algorithm 9 shows Linux self-debugging.

Algorithm 9 Linux Self-Debugging

```

StartFullTime=0           ▷ Full test start time
StartSingleTime=0         ▷ Single test start time
EndFullTime=0             ▷ Full test end time
EndSingleTime=0           ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
computational cost
    StartSingleTime=StartTimer() ▷ Read current time
and store in StartSingleTime
    If ptrace(PTRACE_TRACEME) < 0 ▷ If this passes,
no debugger yet
        NOP                               ▷ Executing a
penalty would skew results, however the check is part
of the technique for measurement
    If ptrace(PTRACE_TRACEME) != -1      ▷ If this
passes, we are debugging ourselves
        NOP                               ▷ Executing a
penalty would skew results, however the check is part
of the technique for measurement
    TimeIntensiveOperations(byte) ▷ Perform math
operations that take time on each byte
    ptrace(PTRACE_DETACH)              ▷ Stop debugging
ourselves
    EndSingleTime=StopTimer() ▷ Read current time
and write times to log file (Single)
EndFullTime=StopTimer() ▷ Read current time and write
times to log file (Full)

```

Algorithm 10 Code Obfuscation

```

StartFullTime=0           ▷ Full test start time
StartSingleTime=0        ▷ Single test start time
EndFullTime=0            ▷ Full test end time
EndSingleTime=0          ▷ Single test end time
StartFullTime=StartTimer() ▷ Read current time and
store in StartFullTime
Foreach byte in array ▷ Use chars in large array as part of
computational cost
    StartSingleTime=StartTimer() ▷ Read current time
    and store in StartSingleTime
    TimeIntensiveOperations_1(byte) ▷ Perform
    operations that take time, part 1 for loop unrolling
    (myfunc+offset)(randomint()) ▷ Call other function
    based on offset (pointer arithmetic)
    TimeIntensiveOperations_2(byte) ▷ Perform
    operations that take time, part 2 for loop unrolling
    TimeIntensiveOperations_3(byte) ▷ Perform
    operations that take time, part 3 for loop unrolling
    Bogus(1,2,3,4,5) ▷ Call bogus function with incorrect
    arguments
    TimeIntensiveOperations_4(byte) ▷ Perform
    operations that take time, part 4 for loop unrolling
    EndSingleTime=StopTimer() ▷ Read current time
    and write times to log file (Single)
EndFullTime=StopTimer() ▷ Read current time and write
times to log file (Full)

```

6) CODE OBFUSCATION

The seventh and last technique will be code obfuscation. We will add a few instances of code obfuscation to our base sample to obfuscate the original code at the source, assembly, and binary level to make the code harder to sign or detect. Code obfuscation may include components such as additional unnecessary instructions, function pointer manipulation, name mangling, and deception. It is worth keeping in mind that our implementation is not equivalent to all code obfuscation implementations, and differences between this implementation and others should be considered when generalizing research results.

For our algorithm, Algorithm 10, we used a couple of code obfuscation components to implement this anti-debugging technique. We unrolled some loops, added bogus function calls, and used pointer arithmetic to call a function without explicitly calling it in the code [22].

The loop unrolling adjusts the loop to increment by 4 and instead runs each of the iterations sequentially within the scope of the loop. In addition, there will be junk calls and math being done to further obfuscate the code from being debugged. The function pointer arithmetic is part of the code obfuscation as well. The function pointer starts by pointing at one function and ends up executing another.

Lastly, we include a bogus function call that will do nothing. The entire point of this anti-debugging technique

is to confuse and hinder any debugging or analysis of this algorithm.

IV. RESULTS

This section presents the results of various tests. The baseline test was conducted for both the Windows and Linux OS systems. The Windows system is measured in ticks, which is approximately 10,000 ticks per millisecond. This means that the `QueryPerformanceCounter()` essentially measures 100 nanosecond blocks of time. In addition, only data within 3 standard deviations (i.e., $\text{abs}(z\text{-score}) < 3$) is kept for this data analysis.

A. BASELINE TEST

The Base test is the first test run and sets the baseline numbers for the rest of the test. While we are not drawing distinct differences between a regular run and an immediate second run with the technique, which will be discussed in future work, the differences in the statistics can give a clear picture of the amount of CPU time a technique needs. The Windows results are in Fig. 1, and the Linux results are in Fig. 2. The statistical data is shown in Table 3. Based on the numbers for the baseline test, very often Windows is faster for this baseline except for the highest 25% of execution times. Using the 100x factor conversion between Ticks and nanoseconds, the Windows baseline ran within 6 Ticks, or 600 nanoseconds, for 75% of the data. The Linux baseline ran within approximately 10 Ticks (10725ns) and 17 Ticks (17185ns) across all the data. The 25% least efficient quickly rises in execution time based on the first standard deviation ending near 800 Ticks of execution time. While Windows seems to have significant bursts of inactivity, Linux performs much more consistently.

TABLE 3. Base test results for Windows and Linux.

Windows		Linux
Metric	Value (Ticks)	Values (Ticks)
Count	998982	995105
Mean	132.925	113.65513
Std	707.81	4.52239
Min	0	107.25
25%	0	113.02
50%	2	113.19
75%	6	113.37
Max	6194	171.85

B. API TEST

The API test was created to test the usage of a system API to check if the process is being debugged. For Windows, this is trivial by using the `IsDebuggerPresent` API call. On Linux, however, there is no such call. As a replacement, we read the `/proc/self/status` file and checked for a non-zero `TracerPid` value. This is very similar to the underlying functionality of `IsDebuggerPresent` when it looks at the second field within the PEB to check if a debugger is attached.

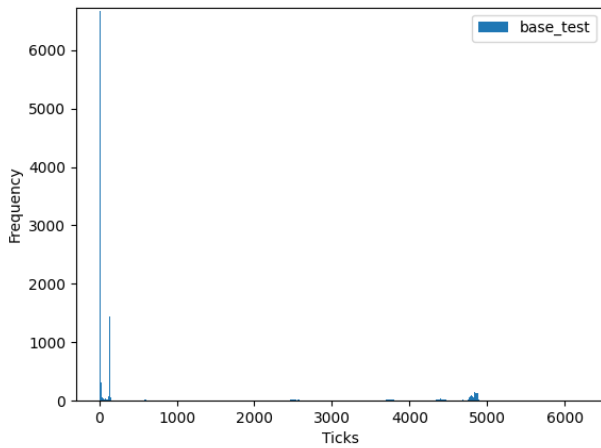


FIGURE 1. Windows base test data.

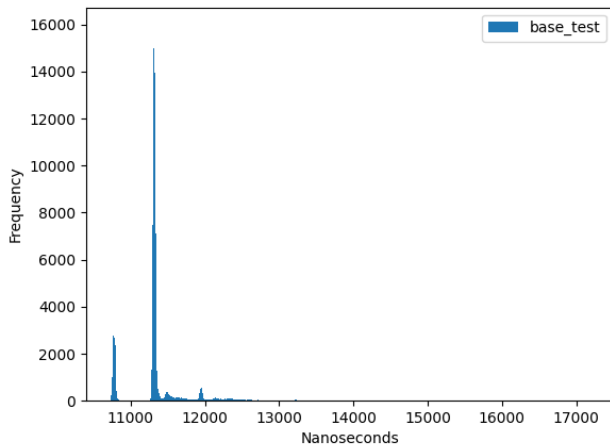


FIGURE 2. Linux base test data.

Since these are not identical operations and there are more Linux computations in this test, we expect to see the Linux execution times be longer than the Windows execution times, and in general, we do see that. The statistical results are shown in Table 4. Figs. 3 and 4 present the Windows and Linux data, respectively. The Windows API test, similar to the baseline, ran in 7 Ticks (700ns) or less for 75% of the runs. The Linux runtimes were significantly longer, ranging from 27 Ticks to 81 Ticks, with an average of 49 Ticks for this operation. However, as we noticed in the baseline, occasionally, Windows is not very performant, and there exists a significant rise in execution time for a small subset of the data. These results line up very well with our expectations for this test. Given that the `IsDebuggerPresent` function consists of approximately three instructions to get the TIB address, get the PEB address, and reference data two bytes into the PEB Structure, a gain of 0-1 Ticks for 75% of the data seems reasonable. In addition, a gain of 16-63 Ticks for having to read and parse a file to acquire the same information on Linux also seems reasonable.

TABLE 4. API test results for Windows and Linux.

Windows		Linux
Metric	Value (Ticks)	Values (Ticks)
Count	999047	991433
Mean	134.19	490.8251
Std	704.964	40.61945
Min	0	268.62
25%	0	464.96
50%	2	485.24
75%	7	509.32
Max	6598	809.94

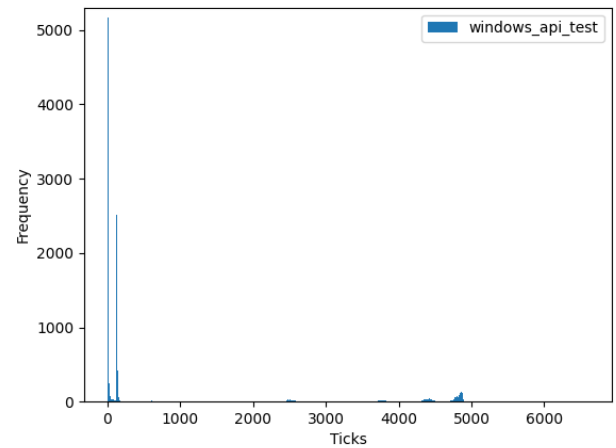


FIGURE 3. Windows API test data.

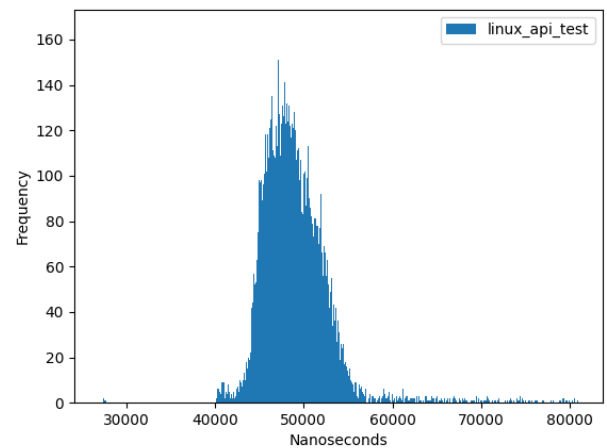


FIGURE 4. Linux API test data.

C. CODE OBFUSCATION TEST

For this test, the code was effectively identical. The minor change was converting from `WriteFile()` to `write()` from Windows to Linux. These functions were not run and should fail quickly, therefore not significantly impacting the code obfuscation timing. The statistical data is available in Table 5. Fig. 5 presents the Windows data, and the Linux results are shown in Fig. 6.

The code obfuscation test is one of the more accurate tests, given the similarity of the code between operating systems.

TABLE 5. Code obfuscation test results for Windows and Linux.

Windows		Linux
Metric	Value (Ticks)	Values (Ticks)
Count	998043	999965
Mean	413.749	1175.3502
Std	1262.467	1260.5885
Min	0	277.22
25%	1	414.72
50%	4	460.14
75%	46	1869.88
Max	11497	4959.30

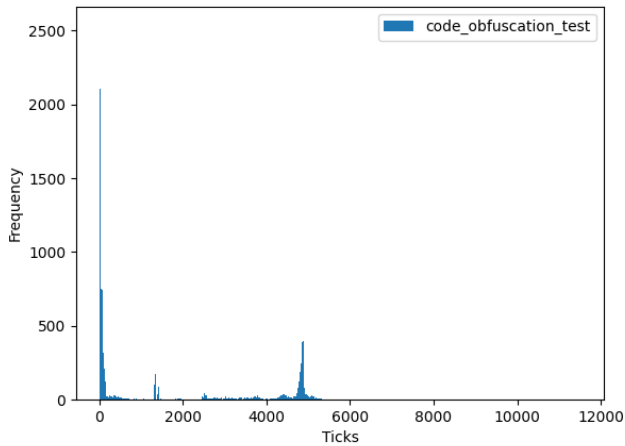


FIGURE 5. Windows code obfuscation test data.

This test will give more insight into efficiencies when it comes to the overhead for lots of arithmetic operations on lots of data. The only major difference between these functions between operating systems is the final bogus function; Windows uses `WriteFile()` with bogus arguments, and Linux uses `write()` with bogus arguments. These functions were added as part of the obfuscation technique and certainly not necessary for executing the code.

We observed the common pattern of more consistent and bound Linux run times compared to Windows' quick execution for 75% of the individual runs. The code obfuscation does take additional time compared to the baseline and previous API test: 75% of the runs on Windows were between 0 and 46 Ticks. Once again, the least efficient 25% quickly jumps in execution time given the standard deviation of over 1000 Ticks. On the Linux side, however, we have an average of approximately 117 Ticks, with the range across all the data between 27 and 495 Ticks. While the actual number for standard deviations is higher in the table, keep in mind the Linux nanoseconds vs. the Windows Ticks. The Linux standard deviation in Ticks is only 126 compared to Windows' 1200+ Ticks standard deviation. Another interesting phenomenon is that Windows and Linux seem to have two major groupings of execution times for this test.

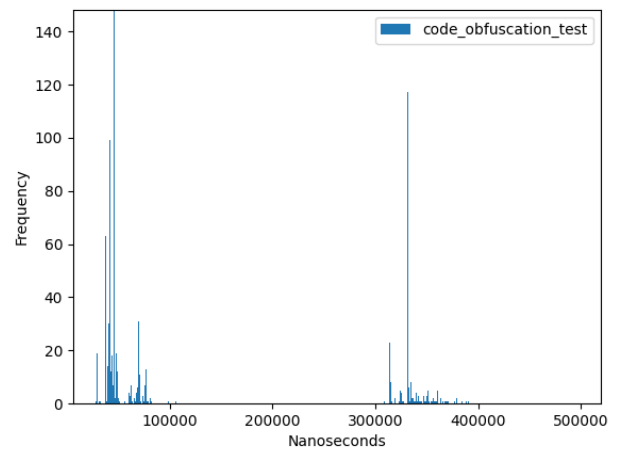


FIGURE 6. Linux code obfuscation test data.

TABLE 6. Memory encryption test results for Windows and Linux.

Windows		Linux
Metric	Value (Ticks)	Values (Ticks)
Count	990335	996056
Mean	6614.587	18595.46
Std	4668.32	613.5787
Min	0	13492.10
25%	4274	18265.47
50%	5386	18394.03
75%	9730	18703.92
Max	58866	24271.52

D. MEMORY ENCRYPTION TEST

This test was the most time-consuming based on how much effort was needed per iteration—the information needed to be decrypted, modified, and re-encrypted, resulting in many additional computations. The Windows results are in Fig. 7, and the Linux results are in Fig. 8. The statistical data is located in Table 6.

The memory encryption test was the most computationally expensive test due to the numerous functions, libraries, and implementation details that needed to be executed to encrypt and decrypt successfully. Windows provides access to built-in encryption functions. However, we ended up using a library from OpenSSL, EVP for Linux. For this test, we only looked at decrypting and encrypting data before and after it is used, respectively, for the test. Another possible test would include decryption, execution, and re-encryption. This would likely not have a major change in the results, but it is an anti-debugging technique to encrypt sensitive information for strictly controlled use. In this case, it is a function rather than data. Given how secure encryption is, it is likely that memory encryption is the most secure anti-debugging technique so long as the key is protected. Once the key has been introduced or is visible in memory, the protection goes away because someone can extract the key once it is in memory, decrypting the important information.

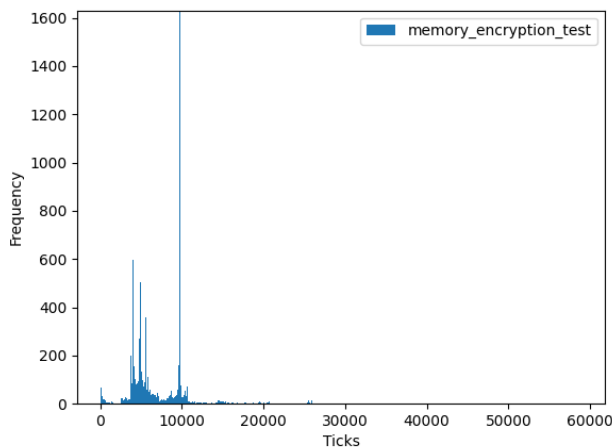


FIGURE 7. Windows memory encryption test data.

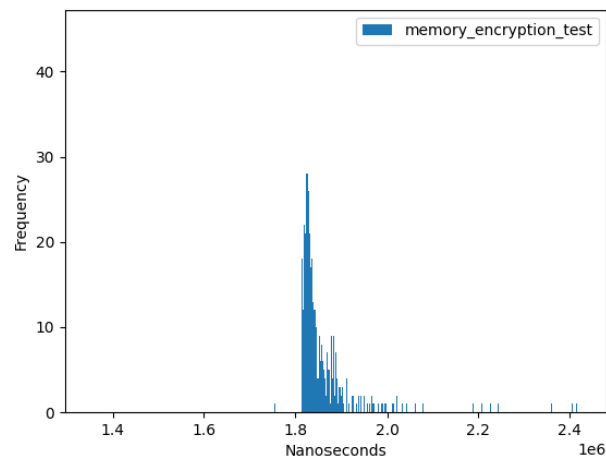


FIGURE 8. Linux memory encryption test data.

E. TIMING ANALYSIS TEST

For this test, the code measures how long execution takes between a set of instructions. If that code takes too long to execute, then it is likely the code is being debugged. This test has lower overhead than some other tests due to fewer API calls and the majority of the technique being a subtraction and comparison operation. The Windows data is in Fig. 9, and the Linux data is in Fig. 10. The statistical data for Windows and Linux is in Table 7.

The timing analysis test is for seeing if the code takes longer than expected to run. If this happens, someone likely used a debugger to slow down or stop the code from executing. When this technique is used, multiple calls to clocks or timers are needed to establish a beginning and end time. The threshold is determined using experience or measurement. For the Windows version, a threshold of 0.5 seconds is used. On the Linux version, a threshold of 1 second is used. Both of these thresholds are sufficiently low enough that a human could not do something meaningful in that amount of time while execution was stopped yet long enough that a slow execution of the code will not falsely trigger it. We can see Windows continues to hover around low

TABLE 7. Timing analysis test results for Windows and Linux.

Windows		Linux
Metric	Value (Ticks)	Values (Ticks)
Count	977228	992757
Mean	28.637	142.7235
Std	153.568	7.0637
Min	0	131.24
25%	0	138.92
50%	2	139.67
75%	6	146.62
Max	3331	224.05

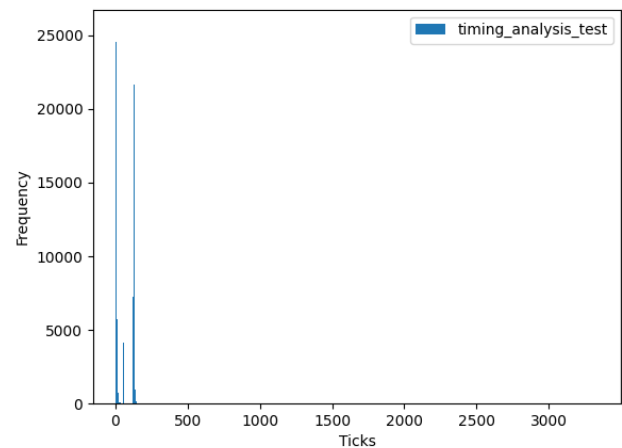


FIGURE 9. Windows timing analysis test data.

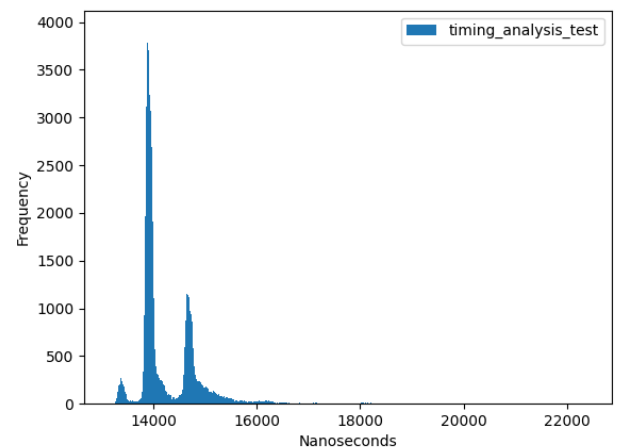


FIGURE 10. Linux timing analysis test data.

Ticks for a majority of the execution attempts. The standard deviation is also fairly low, with it being only 153 Ticks compared to the Code Obfuscation test. Once again, Linux is very consistent with a much smaller range. Linux varies between 13 and 22 Ticks of execution time for each timing analysis test.

F. PARENT PROCESS TEST

This test checks that the parent process of the running process is not a debugger. This requires significantly more API calls for Windows. The Windows data is in Fig. 11, and the Linux

TABLE 8. Parent process test results for Windows and Linux.

Windows		Linux
Metric	Value (Ticks)	Values (Ticks)
Count	999887	992030
Mean	49470.893	192.5095
Std	20442.4243	9.2758
Min	0	176.21
25%	39015	188.14
50%	44210	189.25
75%	53803	193.26
Max	973738	285.17

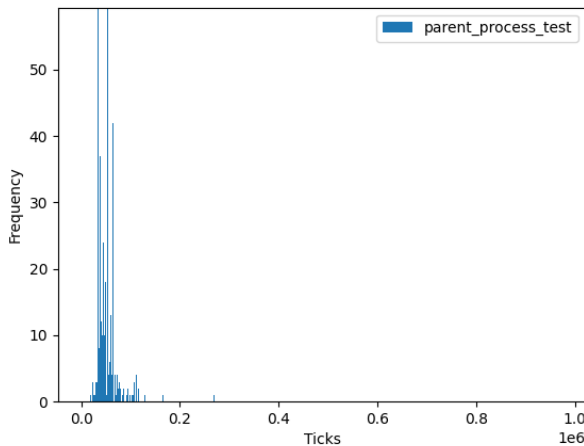


FIGURE 11. Windows parent process test data.

data is in Fig. 11. Table 8 shows the Windows and Linux statistical data.

The parent process test is fairly simple and similarly implemented across platforms; however, the ability to get parent processes and parent image names varies between the platforms, which can account for some of the change in execution time. Windows has significantly more work that needs to be done to get the parent process' image name. Linux can open the `/proc/getppid() /status file, where getppid() returns the parent process id. This is faster and easier than using the CreateToolhelp32Snapshot() and iterating through all of the processes to find the parent. This is represented quite well in the results. On Windows, the execution time varies from 0 to 973 thousand Ticks, with only 25% of the data being less than 39 thousand Ticks for this test. The average is almost 50 thousand Ticks, with a standard deviation of approximately 20 thousand Ticks. Not only is that a lot of time, but it's also a highly variable amount of time to check the name of the parent process on Windows. On Linux, we're consistently within 176 and 286 Ticks of execution time total, with an average of approximately 192 Ticks and a standard deviation of 9 Ticks. The Linux execution time is more tightly bound and faster in a majority of the cases.`

G. SELF-DEBUGGING TEST

This test looks at another technique to prevent debuggers from controlling the process. The Windows version sets up

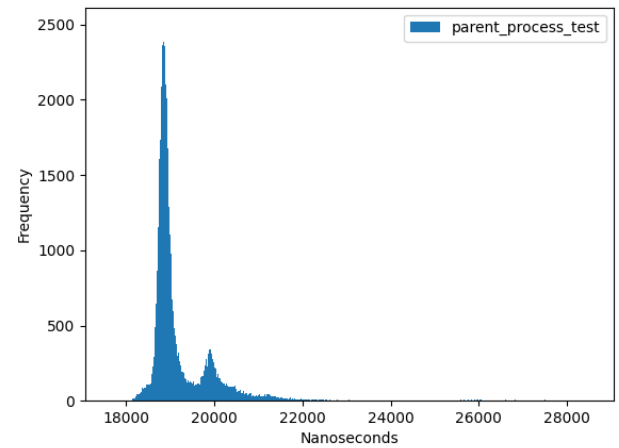


FIGURE 12. Linux parent process test data.

a basic debugger as a child process to debug the parent. The Linux process can debug itself using `ptrace()`. The Windows data is plotted in Fig. 13, and the Linux data is presented in Fig. 14. The statistical data for Windows and Linux is summarized in Table 9.

Lastly, the self-debugging test was another resource-intensive test for the Windows operating system. Based on the specifications for the VM, the test was unreliable beyond approximately 50k runs and caused the system to halt execution effectively. The number of runs for this test for Windows was around 57k, which is a large sample but significantly fewer than other tests, which had data points on the order of one million. In addition, the longer this test ran, the longer the execution times became per test. When looking at the first 50k data points, as an example, the standard deviation of the data changes from millions of ticks down to tens of thousands of ticks. A more capable system is needed to get more data for this type of test. Only the first 50k samples were selected in this test, and the table and graph describe that data. The full data is in [30] for additional analysis. Like all the tests, outliers were removed by removing data beyond three standard deviations from the mean. This outlier removal was done after the selection of the first 50k samples.

On Linux, this test took longer than others, as the average amount of time spent per iteration of this test is approximately 45,000 Ticks (4 ms). The actual range of execution times varied from 38,000 Ticks to over 52,000 Ticks. These are equivalent to varying between 3 and 5ms per iteration.

H. COMPARISON - WINDOWS VERSUS LINUX ANTI-DEBUGGING PERFORMANCE

The baseline results of executing code in user mode on Windows and Linux operating systems showed that Windows seems to cache the results more accurately than Linux. Once the information was loaded into the cache, Windows would execute faster approximately 75% of the time. However, the Linux results were much more consistent and stayed within 107-172 ticks of execution time, while Windows, in the worst

TABLE 9. Self-debugging test results for Windows (50k) and Linux.

Windows		Linux
Metric	Value (Ticks)	Values (Ticks)
Count	49696	992431
Mean	119855	45350.65
Std	75122	165.237
Min	14612	38973.92
25%	69713	44169.80
50%	103105	45065.55
75%	145738	46249.32
Max	813789	52359.96

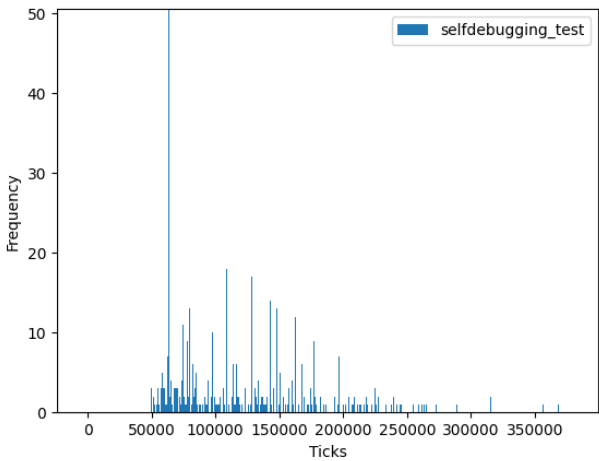


FIGURE 13. Windows self-debugging test data (first 50k).

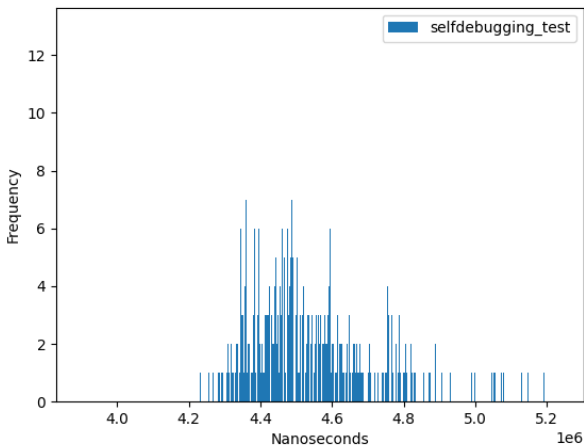


FIGURE 14. Linux self-debugging test data.

case, would take over 6000 ticks. These results may be due to differences in the operating system scheduler or better caching.

The API test results align with expectations as the `IsDebuggerPresent()` call reads a value from memory that's already loaded, while the Linux version has to read and parse a file to find the value. The increase in execution time for the Linux technique matches expectations; with a longer, consistent distribution of execution times while the Windows

technique requires very little execution time with the worst case performing exceptionally slow in comparison.

The code obfuscation results were aligned with expectations for each operating system, however, one surprise emerged with the standard deviation being very close for both techniques. Given that the code obfuscation did not rely heavily on specific API calls and mostly on the code executed as part of the test, we can see the actual impact of the operating system API calls on the actual test. This highlights the impacts of the operating system API calls on anti-debugging technique performance. Otherwise, as expected, Windows optimized via caching or had some results greatly affected by the operating system scheduler and Linux maintained a very consistent distribution of execution times.

The memory encryption tests give additional insight into the impact made by the coding choices of the encryption algorithms. Both algorithms used AES-128 for encryption of the data but the Windows implementation made use of built-in functions while the Linux version used OpenSSL's implementation. There seems to be a correlation between using operating system APIs and an increase in speed of execution of the anti-debugging techniques as we can see in all the previous results. This result is no different as the AES-128 implementation for Linux was slower across the board but maintained a smaller standard deviation. Unlike some of the other Windows results, the Windows encryption API calls did not result in as stark of a difference between the lower 75% and upper 25% of the data.

Once again, the results fall into a skewed distribution for Windows execution time due to either caching or the scheduler. Linux results are within a tight distribution with generally higher overhead compared to Windows except for the worst cases. For this result specifically, both timing distributions seemed to have two peaks in the data.

The parent process results are the first case where Windows has an overall higher execution time than Linux for an anti-debugging technique. While both techniques check the parent process name by getting the parent process pid and checking the string against a list of known bad parent process names, the Linux implementation of reading the `/proc/`getppid()`/status` file is significantly faster than iterating over the results from `CreateToolhelp32Snapshot()` in Windows.

Lastly, the self-debugging technique is the only technique measured in this research that requires another process. This will significantly increase the amount of time needed to execute the anti-debugging technique as it requires creating a child process, attaching to the parent, and blocking the child process when starting and stopping the child process. We can see that overhead in the execution time results as this test has some of the largest values of the entire paper. We still see the skewed distribution in Windows as the largest execution times are at least an order of magnitude larger than the fastest and Linux execution times still have a tighter distribution as indicated by a smaller standard deviation but overall the

execution time takes significantly longer for both operating systems.

V. CONCLUSION AND FUTURE WORK

A. CONCLUSIONS

This paper proposed novel research by measuring different anti-debugging techniques and the CPU time/cost for execution compared to a baseline program. Some of the tests are API calls, memory encryption, timing analysis, or code obfuscation techniques. These differences can be quantified and used for other research or to make informed decisions about protection features for the software. Our measurement of the overhead CPU time may influence which anti-debugging technique will be included based on the goal of the software. For instance, some malware may want minimal CPU overhead to prevent execution time from increasing. From another perspective, software publishers might desire, or require, the use of high-overhead, high-assurance memory encryption. In addition, while not demonstrated in this research, each technique can be used alone or in conjunction with other techniques and have additional amounts of overhead and effectiveness.

The overall impact of measuring these anti-debugging execution times gives us the first inkling of developing a profile for the execution time of these anti-debugging techniques. In general, the Windows distribution is highly skewed. The understanding of why the data is skewed is outside the scope of this paper but is likely impacted by the caching or operating system scheduling algorithms that differ between each operating system. Linux anti-debugging techniques tend to have a smaller standard deviation, which would give a different timing profile compared to Windows techniques, which implement the same type of anti-debugging technique. These profiles can be developed in future work to detect the anti-debugging technique within a program by looking at the differences in execution times or could be used to inform which anti-debugging techniques to include as part of a software protection framework to optimize, in some manner, the overhead execution time for the protected software.

B. FUTURE WORK

Anti-debugging in all its forms and applications will not disappear from the cyber industry anytime soon. These techniques hinder professionals and amateurs alike from fully understanding the internal details of programs while still allowing users to use the programs with minimal disruption. Based on the current landscape, issues like Right-to-Repair and the DMCA laws will keep copy-protection codes around and available for all kinds of uses. Similarly, but less legal, malware will continue using these techniques to prevent understanding, evade detection, and increase the number of victims for any deployment. If the analysts cannot find a way to identify malware, it will be very difficult to detect it and, therefore, block it from a system.

There still remains more to be explored and understood with respect to anti-debugging. Additional research into new techniques, new anti-debugging avoidance measures, usage, prevalence, or even additional types of measurement for better categorization of techniques. For example, future work could be a) an analysis across architectures for these techniques. Architectures can have different use cases, registers, and limitations and there may exist anti-debugging checks that exist in one architecture that do not exist in another based on a particular implementation discrepancy or architecture feature; b) as discussed in section V-A, execution timing profiles can be developed for each technique and for each operating system to inform users of the execution impacts of implementing these techniques within their code. These profiles could be used for detection based on changes in the execution timings or as decisions to optimize software protection frameworks; c) this paper only focuses on using VMWare Workstation as the virtualization platform; the performance of anti-debugging techniques could vary based on the hypervisor in use for testing, especially type-1 versus type-2 hypervisors; d) measuring performance in user-mode can lead to more realistic but less accurate results in terms of raw performance. Follow-on research that implements a kernel-mode driver could force execution to execute only on a single thread with no impact from the operating system scheduler. This would decrease the variance of the timings, but is less representative of the timings of anti-debugging techniques in use in the wild.

REFERENCES

- [1] S. Cesare. (1999). *Linux Anti-Debugging Techniques (Fooling the Debugger)*. [Online]. Available: <http://www.ouah.org/linux-anti-debugging.txt>
- [2] J. Xu, L. Zhang, L. Yang, Y. Mao, and X. Shi, "An effective Android software reinforcement scheme based on online key," in *Proc. IEEE 18th Int. Conf. High Perform. Comput. Commun., IEEE 14th Int. Conf. Smart City, IEEE 2nd Int. Conf. Data Sci. Syst. (HPCC/SmartCity/DSS)*, Dec. 2016, pp. 1544–1548.
- [3] Microsoft. *IsDebuggerPresent Function (Debugapi.h)*. Accessed: Feb. 2025. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/debugapi/nf-debugapi-isdebuggerpresent>
- [4] J.-W. Kim, J. Bang, Y.-S. Moon, and M.-J. Choi, "Disabling anti-debugging techniques for unpacking system in user-level debugger," in *Proc. Int. Conf. Inf. Commun. Technol. Conver. (ICTC)*, Oct. 2019, pp. 954–959.
- [5] L. Guoyuan, T. Jiutao, and G. Bing, "A survey on shareware protection schemes," in *Proc. 2nd Int. Conf. Signal Process. Syst.*, vol. 3, Jul. 2010, p. 697.
- [6] A. M. Bahaa-Eldin and M. A. A. Sobh, "A comprehensive software copy protection and digital rights management platform," *Ain Shams Eng. J.*, vol. 5, no. 3, pp. 703–720, Sep. 2014.
- [7] M.-J. Kim, J.-Y. Lee, H.-Y. Chang, S. Cho, Y. Park, M. Park, and P. A. Wileyse, "Design and performance evaluation of binary code packing for protecting embedded software against reverse engineering," in *Proc. 13th IEEE Int. Symp. Object/Compon./Service-Oriented Real-Time Distrib. Comput.*, May 2010, pp. 80–86.
- [8] K. Yoshizaki and T. Yamauchi, "Malware detection method focusing on anti-debugging functions," in *Proc. 2nd Int. Symp. Comput. Netw.*, Dec. 2014, pp. 563–566.
- [9] H. Chang, H.-J. Kwon, and I. You, "The study on software tamper resistance for securing a game service," in *Proc. Int. Conf. Complex, Intell. Softw. Intensive Syst.*, Mar. 2009, pp. 555–558.
- [10] C. Lim and Nicsen, "Mal-EVE: Static detection model for evasive malware," in *Proc. 10th Int. Conf. Commun. Netw. China (ChinaCom)*, Aug. 2015, pp. 283–288.

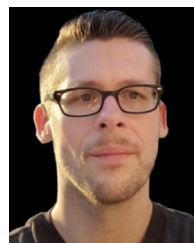
- [11] X. Chen, J. Andersen, Z. M. Mao, M. Bailey, and J. Nazario, "Towards an understanding of anti-virtualization and anti-debugging behavior in modern malware," in *Proc. IEEE Int. Conf. Dependable Syst. Netw. With FTCS DCC (DSN)*, Jun. 2008, pp. 177–186.
- [12] H. Shi, J. Mirkovic, and A. Alwabel, "Handling anti-virtual machine techniques in malicious software," *ACM Trans. Privacy Secur.*, vol. 21, no. 1, pp. 1–31, Feb. 2018.
- [13] J. Aycock, J. M. G. Cardenas, and D. M. N. de Castro, "Code obfuscation using pseudo-random number generators," in *Proc. Int. Conf. Comput. Sci. Eng.*, 2009, pp. 418–423.
- [14] G. Paoloni. (2010). *How To Benchmark Code Execution Times on Intel IA-32 and IA-64 Instruction Set Architectures*. [Online]. Available: <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/ia-32-ia-64-benchmark-code-execution-paper.pdf>
- [15] M. N. Gagnon, S. Taylor, and A. K. Ghosh, "Software protection through anti-debugging," *IEEE Secur. Privacy Mag.*, vol. 5, no. 3, pp. 82–84, May 2007.
- [16] T. Shields, "Anti-debugging—A developers view," Veracode, Burlington, MA, USA, Tech. Rep., 2010.
- [17] H. Shi and J. Mirkovic, "Hiding debuggers from malware with apate," in *Proc. Symp. Appl. Comput.*, Apr. 2017, pp. 1703–1710.
- [18] A. Li, Y. Zhang, J. Zhang, and G. Zhu, "A token strengthened encryption packer to prevent reverse engineering PE files," in *Proc. Int. Conf. Estimation, Detection Inf. Fusion (ICEDIF)*, Jan. 2015, pp. 307–312.
- [19] T. Yi, A. Zong, M. Yu, S. Gao, Q. Lin, P. Yu, Z. Ren, and Z. Qi, "Anti-debugging framework based on hardware virtualization technology," in *Proc. Int. Conf. Res. Challenges Comput. Sci.*, Dec. 2009, pp. 218–220.
- [20] B. Abrath, B. Coppens, I. Nevolin, and B. De Sutter, "Resilient self-debugging software protection," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops*, Sep. 2020, pp. 606–615.
- [21] T. Holl, P. Klocke, F. Franzen, and J. Kirsch, "Kernel-assisted debugging of Linux applications," in *Proc. 2nd Reversing Offensive-Oriented Trends Symp.* New York, NY, USA: Association for Computing Machinery, Nov. 2018, pp. 1–9.
- [22] J. Gan, R. Kok, P. Kohli, Y. Ding, and B. Mah, "Using virtual machine protections to enhance whitebox cryptography," in *Proc. IEEE/ACM 1st Int. Workshop Softw. Protection*, May 2015, pp. 17–23.
- [23] J. Wan, M. Zulkernine, and C. Liem, "A dynamic app anti-debugging approach on Android ART runtime," in *Proc. IEEE 16th Int. Conf. Dependable, Autonomic Secure Comput., 16th Int. Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol. Congress (DASC/PiCom/DataCom/CyberSciTech)*, Aug. 2018, pp. 560–567.
- [24] G. Bing, L. Guoyuan, and T. Jiutao, "Application of structured exception handling in software anti-debugging," in *Proc. 3rd Int. Conf. Adv. Comput. Theory Engineering (ICACTE)*, vol. 2, Aug. 2010, p. 133.
- [25] S. Choi, T. Chang, C. Kim, and Y. Park, "X64Unpack: Hybrid emulation unpacker for 64-bit windows environments and detailed analysis results on VMProtect 3.4," *IEEE Access*, vol. 8, pp. 127939–127953, 2020.
- [26] J. Lee, B. Kang, and E. G. Im, "Rule-based anti-anti-debugging system," in *Proc. Res. Adapt. Convergent Syst.*, Oct. 2013, pp. 353–354.
- [27] J. Park, Y.-H. Jang, S. Hong, and Y. Park, "Automatic detection and bypassing of anti-debugging techniques for Microsoft windows environments," *Adv. Electr. Comput. Eng.*, vol. 19, no. 2, pp. 23–28, 2019.
- [28] J. Voisin, J. Crowell, and A. Possemato, "Jvoisin/Pangu: Toolkit to detect/crash/attack GNU debugging-related tools," Apr. 2016. [Online]. Available: <https://github.com/jvoisin/pangu>
- [29] B. Day, "Hacker's corner: Complete guide to anti-debugging," Apr. 2024. [Online]. Available: <https://linuxsecurity.com/features/anti-debugging-for-noobs-part-1>
- [30] Quantumite. *Quantumite/AntidebuggingResearch: Collection of Resources for Anti-Debugging Research*. Accessed: 2023. [Online]. Available: <https://github.com/Quantumite/AntidebuggingResearch>



AUSTIN NORBY received the Ph.D. degree in cyber operations from Dakota State University, Madison, SD, USA. He is the Director of Internal Research and Development, Bogart Associates of Northern Virginia. His research interests include offensive security, anti-debugging techniques, operating systems security, and reverse engineering.



BHASKAR P. RIMAL (Senior Member, IEEE) is an Assistant Professor with the Department of Computer Science, University of Idaho, Moscow, USA. He was an Assistant Professor with Dakota State University, Madison, SD, USA. He was a Visiting Assistant Professor and an Academic Specialist with the Department of Computer Science, Tennessee Technological University, Cookeville, TN, USA. He was a Visiting Scholar with the Department of Computer Science, Carnegie Mellon University, Pittsburgh, PA, USA. He is a Senior Member of ACM and a Senior Professional Member of ASEE. He currently serves as an Associate Editor for IEEE ACCESS and IEEE TRANSACTIONS ON TECHNOLOGY AND SOCIETY. Further, he served as the Technical Editor for *IEEE Network Magazine*, from 2021 to 2024; an Associate Technical Editor for *IEEE Communications Magazine*, from 2017 to 2020, and *EURASIP Journal on Wireless Communications and Networking*, from 2017 to 2023; and an Editor for *Internet Technology Letters*.



BRAMWELL BRIZENDINE is an Assistant Professor with The University of Alabama in Huntsville. He specializes in software exploitation, code-reuse attacks, reverse engineering, shellcode analysis, and malware analysis. He presented his research at many top cybersecurity and hacking conferences.

...